



Univerzitet u Novom Sadu
Tehnički fakultet "Mihajlo Pupin"

Zrenjanin



Slojevita arhitektura operativnog sistema -seminarski rad-

Predmet: Operativni sistemi

Profesor: Prof. dr Ljubica Kazi
Asistent: Msc Vuk Amizic

Student: Nikola Ignjatov
Broj indeksa: SI 49/22

Zrenjanin, 2026.

SADRŽAJ

Uvod.....	3
1 Teorijski deo	3
1.1 Teorijske osnove	3
1.2 Tehnološke osnove.....	4
1.3 Komparativna analiza postojećih rešenja	4
1.4 Primeri primene.....	4
2 Praktični deo	5
2.1 Namena softvera i korisničko uputstvo	5
2.1.1 Pokretanje i osnovne opcije	5
2.1.2 Primer uspešne obrade.....	6
2.1.3 Dozvole i obrada grešaka	7
2.2 Kratak opis alata i tehnologija.....	8
2.3 Opis implementacije.....	8
2.3.1 Strukture i povezana lista	8
2.3.2 Odgovornost slojeva	9
2.3.3 Datoteke i provera unosa	9
2.4 Provera rada i ograničenja.....	10
Zaključak.....	11
Literatura.....	12

UVOD

Slojevita arhitektura predstavlja način organizacije operativnog sistema u celine sa jasnim odgovornostima i međusobnim granicama. Viši sloj traži uslugu od nižeg, a rezultat se vraća u suprotnom smeru. Ovaj rad objašnjava osnovni model, prikazuje kako se ideja koristi u stvarnim sistemima i daje obrazovnu simulaciju prolaska zahteva kroz četiri sloja. Simulacija nije stvarni operativni sistem: ona ilustruje tok poziva i odvajanje odgovornosti.

1 TEORIJSKI DEO

1.1 Teorijske osnove

Operativni sistem upravlja procesorom, memorijom i ulazno-izlaznim uređajima i programima pruža interfejs za korišćenje tih resursa. Tipične oblasti su procesi, memorija, konkurentnost i trajno čuvanje podataka [1]. Slojevita organizacija razdvaja te oblasti kroz interfejse: svaki sloj skriva deo implementacije od sloja iznad njega [2].

Slojevi predstavljaju nivoe apstrakcije. U modelu iz ovog rada korisnički interfejs predaje zahtev sistemskim uslugama, koje proveravaju dozvolu i pronalaze blok. Drajver proverava dostupnost uređaja, a model diska izvršava operaciju. To je obrazovna organizacija; stvarni sistemi imaju drugačije granice i brojeve slojeva. Apple, na primer, opisuje više tehnoloških nivoa od jezgra i upravljačkih programa do usluga i aplikacija [2].

Prednost slojeva je jasnija podela odgovornosti: promena implementacije nižeg sloja ne mora menjati korisnika njegovog stabilnog interfejsa. To olakšava održavanje, testiranje i prenos na drugi hardver. Cena može biti dodatni prolaz kroz interfejse i teže projektovanje granica kada dve usluge moraju blisko da sarađuju. Zbog toga je slojevitost projektantski princip, a ne garancija da je svaki stvarni kernel strogo slojevit [3].

Istorijski primer je sistem THE, opisan u Dijkstrinom radu iz 1968. godine. Njegovi nivoi obuhvatali su raspodelu procesora, upravljanje segmentima memorije, komunikaciju sa operaterom, ulazno-izlazno baferovanje i korisničke programe; iznad njih razmatran je operater. Niži nivo uvodio je apstrakciju na kojoj su se gradili viši nivoi. Autor opisuje i testiranje od nižih ka višim nivoima, tako da se novi sloj dodaje nakon provere prethodnog [11].

Dobra granica sloja ne određuje se samo redosledom izvršavanja. Parnas naglašava skrivanje projektantskih odluka koje se mogu menjati: korisnik modula poznaje njegove operacije, dok raspored podataka ostaje unutar modula [12]. U operativnom sistemu to znači da aplikacija može tražiti čitanje datoteke bez poznavanja rasporeda blokova. U priloženoj simulaciji

odvajanje je logičko, kroz funkcije; zajednički niz disk nije potpuno sakriven, što predstavlja namerno pojednostavljenje za mali C projekat.

Logički sloj i režim rada procesora nisu isto. Više slojeva može raditi u istom adresnom prostoru, pa sama podela na funkcije ne sprečava pogrešan pristup memoriji. Korisnički i jezgrski režim predstavljaju posebnu granicu privilegija [5]. Simulacija ne menja režim procesora: svi njeni slojevi izvršavaju se kao obične C funkcije u jednom procesu.

1.2 Tehnološke osnove

Granica između korisničkog i jezgrskog režima pokazuje zašto su interfejsi važni. Korisnički program nema neposredan pristup svim memorijskim adresama i uređajima; kada mu je potrebna zaštićena usluga, koristi sistemski interfejs. Microsoft opisuje razliku između korisničkog i jezgrskog režima i izolaciju procesa [5].

Sloj apstrakcije hardvera (HAL) skriva deo razlika između računarskih konfiguracija od ostalih delova Windows sistema [4]. I/O menadžer prosleđuje ulazno-izlazne zahteve odgovarajućim upravljačkim programima, čime aplikacija ne mora da poznaje fizičku realizaciju uređaja [6]. To je konkretan primer razdvajanja interfejsa od implementacije.

U Linuxu virtuelni sistem datoteka (VFS) pruža zajednički interfejs programima i omogućava istovremeno postojanje različitih implementacija sistema datoteka [7]. Upravljanje virtuelnom memorijom skriva fizički raspored memorije od programa i podržava zaštitu i kontrolisano deljenje [8]. Model uređaja uređuje odnose između uređaja, magistrala i upravljačkih programa [9]. Ovi podsistemi imaju slojevite interfejse, iako ceo Linux kernel nije stroga školska piramida slojeva.

1.3 Komparativna analiza postojećih rešenja

Sistem	Primer slojevitosti	Napomena
Windows	Korisnički režim, sistemske usluge, I/O menadžer, drajveri i HAL [4][5][6]	Drajveri mogu biti raspoređeni u više nivoa [10].
Linux	Korisnički pozivi, VFS, sistemi datoteka i drajveri [7][9]	Podsistemi imaju interfejs; jezgro nije strogo slojevito.
macOS	Više tehnoloških slojeva i kernel sa Mach, BSD i I/O Kit delovima [2][3]	Hibridna organizacija kombinuje više tradicija.

Razlika između primera je u tome gde se postavljaju granice i koje se usluge nalaze na kom nivou. Windows dokumentacija posebno opisuje slojeve drajvera [10]. Apple opisuje XNU kao spoj Mach, BSD i I/O Kit komponenti [3]. Poređenje zato treba da bude usmereno na tok zahteva i ugovore između delova, a ne na tvrdnju da su sva tri sistema izgrađena po istoj šemi.

1.4 Primeri primene

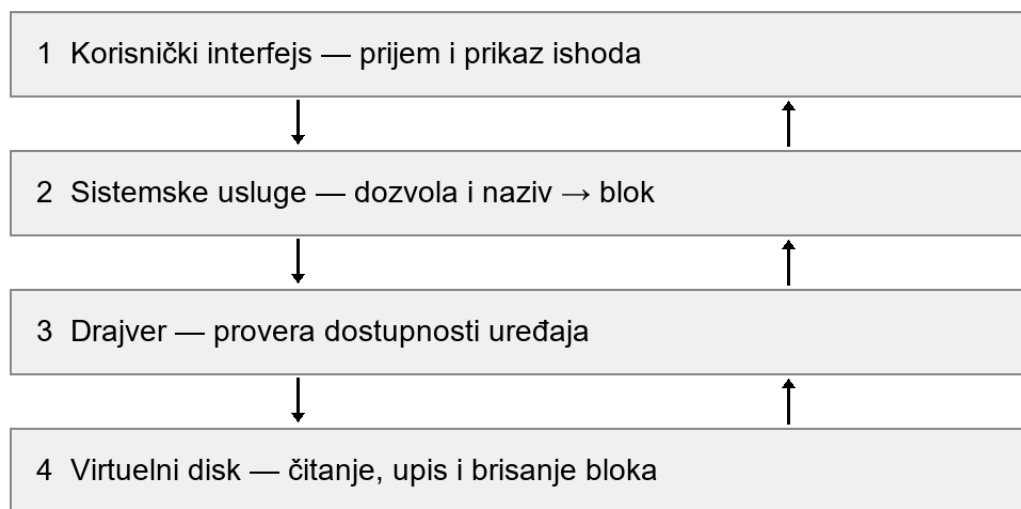
Prilikom čitanja datoteke aplikacija šalje zahtev kroz programski interfejs. VFS ili ekvivalentna sistemska usluga pronalazi odgovarajući sistem datoteka, a niži nivoi mogu komunicirati sa drajverom uređaja. Aplikacija dobija sadržaj bez poznavanja načina rada konkretnog diska [6][7].

Kod rada sa različitim uređajima korisnički program može zadržati isti interfejs dok se menjaju niži upravljački programi. Windows dokumentacija navodi da viši drajveri mogu obrađivati zahtev i proslediti ga nižim drajverima [10]. Slična ideja važi i za apstrakciju memorije: program radi sa virtuelnim adresama, dok sistem i procesor obavljaju mapiranje u fizičku memoriju [8].

2 PRAKTIČNI DEO

2.1 Namena softvera i korisničko uputstvo

Softver je obrazovna simulacija sa četiri logička sloja. Korisnik pravi zahteve za čitanje, upis ili brisanje datoteke. Zahtevi čekaju u jednostruko spregnutoj listi i obrađuju se po FIFO pravilu: prvi dodat zahtev obrađuje se prvi. Virtuelni disk ima pet blokova, a svaka datoteka zauzima jedan blok. Upis menja sadržaj bloka, čitanje prikazuje sačuvani tekst, a brisanje oslobađa blok.



Slika 1. Tok zahteva naniže i povratak statusa naviše. Izvor: dijagram izrađen za ovu simulaciju.

Sistemske usluge proveravaju dozvolu pristupa i prevode naziv datoteke u broj bloka. Drajver može odbiti operaciju ako je uključen simulirani kvar. Rezultat svakog nižeg poziva vraća se pozivaocu, pa korisnik vidi uspeh ili grešku i nivo na kome je zahtev zaustavljen. Prikaz celog diska i čuvanje slike simulacije su administrativne opcije izvan ovog toka zahteva.

2.1.1 Pokretanje i osnovne opcije

Raspakovati Program_CodeBlocks.zip u običan folder, otvoriti SimulacijaSlojeva.cbp u Code::Blocks i izabrati Build and run (F9). Projekat koristi GNU GCC kompajler i standard C11. Ako kompajler nije automatski pronađen, u Settings > Compiler > Toolchain executables izabrati njegov folder. Na računaru na kome je program testiran putanja je C:\Program Files\CodeBlocks\MinGW. Putanja može biti drugačija na drugom računaru.

```
SIMULACIJA SLOJEVITE ARHITEKTURE OPERATIVNOG SISTEMA
Virtuelni disk: 5 blokova; jedna datoteka zauzima jedan blok.

1 Dodaj zahtev      2 Prikazi red      3 Obradi red
4 Prikazi disk      5 Sacuvaj disk     6 Ucitaj disk
7 Rezim pristupa    8 Stanje uredjaja  0 Izlaz
Rezim: citanje i upis | Uredjaj: ispravan
```

Slika 2. Početni ekran. Izvor: stvarni izlaz testiranog programa, formatiran za čitljivost u dokumentu.

Opcija	Značenje
1	Dodavanje zahteva: 1 čitanje, 2 upis, 3 brisanje.
2	Prikaz zahteva koji čekaju u redu.
3	Obrada celog reda i uklanjanje obrađenih zahteva.
4	Prikaz svih blokova i zauzetosti diska.
5	Čuvanje slike diska u virtuelni_disk.txt.
6	Učitavanje slike diska, ako je red prazan.
7	Promena režima između punog pristupa i samo čitanja.
8	Uključivanje ili isključivanje simuliranog kvara uređaja.
0	Izlaz; disk se ne čuva automatski.

2.1.2 Primer uspešne obrade

Izabрати opciju 1, zatim vrstu 2, naziv beleška i sadržaj Operativni sistemi. Ponovo izabрати opciju 1, vrstu 1 i isti naziv. Opcija 2 prikazuje upis pa čitanje. Tek izborom opcije 3 dolazi do izvršavanja: prvi zahtev pravi datoteku, a drugi čita upravo upisani sadržaj. Samo dodavanje u red ne menja disk.

```
[1 Interfejs] Zahtev 2: CITANJE beleška
[2 Sistemske usluge] Provera dozvole i pronalazenje datoteke.
[2 Sistemske usluge] beleška -> blok 0.
[3 Drajver] Provera uredjaja.
[4 Disk] Pristup bloku 0.
[4 Disk] Procitano: Operativni sistemi
[3 Drajver] Povratak: uspeh.
[2 Sistemske usluge] Povratak: uspeh.
[1 Interfejs] Ishod: USPEŠNO.
```

Slika 3. Čitanje prethodno upisanog sadržaja kroz sva četiri sloja. Izvor: stvarni izlaz programa, formatiran za dokument.

```
--- VIRTUELNI DISK ---
Blok 0 | beleška | Operativni sistemi
Blok 1 | slobodan
Blok 2 | slobodan
Blok 3 | slobodan
Blok 4 | slobodan
Zauzeto: 1/5 blokova.
```

Slika 4. Stanje diska posle upisa i čitanja. Izvor: stvarni izlaz programa, formatiran za dokument.

Ponovni upis na naziv beleška menja sadržaj istog bloka. Za brisanje dodati zahtev vrste 3 sa tim nazivom i obraditi red. Posle brisanja čitanje istog naziva daje grešku. Za čuvanje podataka izabрати opciju 5. Nakon ponovnog pokretanja opcija 6 učitava virtuelni_disk.txt iz radnog foldera projekta. Zahtevi u redu se ne čuvaju između pokretanja.

2.1.3 Dozvole i obrada grešaka

Opcija 7 uključuje režim samo čitanja. Upis i brisanje tada se zaustavljaju u drugom sloju, a čitanje postojeće datoteke ostaje dozvoljeno. Opcija 8 uključuje kvar uređaja: zahtev sa ispravnim nazivom i dozvolom stiže do drajvera, ali ne pristupa bloku. Režim i kvar proveravaju se u trenutku obrade reda. Neuspešan zahtev se uklanja iz reda i za ponovni pokušaj mora biti dodat ponovo.

```
[1 Interfejs] Zahtev 2: UPIS a
  [2 Sistemske usluge] Provera dozvole i pronalazenje datoteke.
  [2 Sistemske usluge] GRESKA: dozvoljeno je samo citanje.
[1 Interfejs] Ishod: NEUSPESNO.

[1 Interfejs] Zahtev 1: UPIS a
  [2 Sistemske usluge] Provera dozvole i pronalazenje datoteke.
  [2 Sistemske usluge] a -> blok 0.
  [3 Drajver] Provera uredjaja.
  [3 Drajver] GRESKA: uredjaj nije dostupan.
  [2 Sistemske usluge] Povratak: greska.
[1 Interfejs] Ishod: NEUSPESNO.
```

Slika 5. Odbijanje upisa zbog dozvole i zbog kvara uređaja u odvojenim pokretanjima. Izvor: stvarni izlazi testiranja, formatirani za dokument.

Naziv sadrži najviše 23 ASCII znaka: slova, cifre, tačku, crticu ili donju crtu. Tekst je neprazan, do 79 bajtova i bez separatora |. Za demonstraciju se koriste ASCII slova. Predugačak unos se odbacuje i traži ponovo. Pun disk odbija novi naziv, ali dozvoljava izmenu postojeće datoteke. Neispravan format slike diska ne zamenjuje prethodno stanje u memoriji.

2.2 Kratak opis alata i tehnologija

Rešenje koristi C11 i standardne biblioteke `stdio.h`, `stdlib.h`, `string.h`, `errno.h` i `ctype.h`. Code::Blocks projekat je podešen za GNU GCC i konzolnu aplikaciju. Program je proveren na Windowsu sa MinGW GCC 14.2.0. Prevođenje sa `-std=c11 -Wall -Wextra -Wpedantic -Werror` završeno je bez upozorenja i grešaka. Funkcionalne provere pokretale su stvarni izvršni program sa pripremljenim ulazom; time je testirana C aplikacija, dok Code::Blocks ostaje razvojno okruženje za otvaranje i pokretanje projekta.

Datoteka `simulacija_slojeva.c` sadrži ceo program radi lakšeg praćenja na odbrani. `SimulacijaSlojeva.cbp` čuva podešavanja projekta. `virtuelni_disk.txt` nastaje pri izboru opcije 5. Dokumentacija i prikazi koriste srpsku latinicu, a identifikatori i konzolne poruke srpske reči bez dijakritika radi jednostavnijeg prikaza u konzoli.

2.3 Opis implementacije

2.3.1 Strukture i povezana lista

Zahtev sadrži redni broj, vrstu operacije, naziv datoteke, tekst za upis i pokazivač sledeci. Pokazivač glava označava početak liste, a rep njen kraj. Funkcija `dodaj_zahtev` rezerviše memoriju pozivom `calloc`, proverava unos i vezuje novi element na kraj. Dodavanje preko repa ne zahteva prolazak kroz celu listu.

```
typedef struct Zahtev {
    int broj;
    int vrsta; /* 1 citanje, 2 upis, 3 brisanje */
    char naziv[DUZINA_NAZIVA];
    char tekst[DUZINA TEKSTA];
    struct Zahtev *sledeci;
} Zahtev;
```

Obrada uzima element sa glave, izvršava zahtev, pomera glavu i oslobađa memoriju. Po završetku rep postaje NULL. Preostali neobrađeni elementi oslobađaju se i pri izlasku iz programa. Time lista služi kao stvarni red operacija, a ne samo kao spisak prikazanih podataka.

```
while (glava) {
    tekuci = glava;
    sloj_korisnickog_interfejsa(tekuci);
    glava = tekuci->sledeci;
    free(tekuci);
}
rep = NULL;
```

2.3.2 Odgovornost slojeva

Sloj	Funkcija	Odgovornost
1	sloj_korisnickog_interfejsa	Prikazuje zahtev, poziva sistemsku uslugu i prikazuje konačan ishod.
2	sloj_sistemskih_usluga	Proverava dozvolu, pretražuje nazive i bira blok.
3	sloj_drajvera	Proverava kvar i prosleđuje pristup disku.
4	sloj_diska	Čita, menja ili briše sadržaj konkretnog bloka.

Sistemske usluge petljom for traže postojeći naziv i prvi slobodan blok. Za upis postojećeg naziva koristi se njegov blok. Za novi naziv koristi se slobodan blok. Ako nijedan ne postoji, vraća se neuspeh. Čitanje i brisanje zahtevaju postojeću datoteku. Ova provera prethodi pozivu drajvera.

```
if (kvar_uredjaja) {  
    puts(" [3 Drajver] GRESKA: uredjaj nije dostupan.");  
    return 0;  
}  
uspeh = sloj_diska(zahtev, blok);
```

Povratna vrednost 1 označava uspeh, a 0 neuspeh. Ako drajver otkrije kvar, disk se ne poziva i sadržaj ostaje isti. Upis u sloju diska kopira već provereni naziv i tekst u niz Blok. Brisanje nulira izabrani blok, pa prazan naziv ponovo označava slobodno mesto. strepy se ovde koristi sa izvorima koji su prethodno ograničeni veličinama odgovarajućih polja.

2.3.3 Datoteke i provera unosa

Čuvanje zapisuje zaglavlje SLOJEVITI_DISK_1 i red za svaki zauzeti blok u formatu broj|naziv|tekst. Na primer, 0|beleska|Operativni sistemi označava sadržaj bloka 0. To je stvarna tekstualna datoteka na računaru. Nasuprot tome, operacije nad virtuelnom datotekom beleska rade nad nizom u memoriji. Opcije 5 i 6 predstavljaju čuvanje i obnovu celog modela, pa na njih ne utiču simulirani kvar i režim pristupa.

```
FILE *datoteka = fopen("virtuelni_disk.txt", "w");
/* Za svaki zauzeti blok: */
fprintf(datoteka, "%d|%s|%s\n", mesto,
        disk[mesto].naziv, disk[mesto].tekst);
fclose(datoteka);
```

Ovaj skraćeni izvod prikazuje osnovne pozive; kompletna funkcija proverava otvaranje, povratnu vrednost fprintf i fclose. Učitavanje proverava zaglavlje, opseg blokova, duplikate naziva i blokova, dužine i separatore. Zapisi se prvo smeštaju u lokalni niz novi_disk; tek ispravna cela datoteka kopira se u disk. Čuvanje direktno prepisuje izlaznu datoteku i nije transakcijsko: nagli prekid tokom snimanja može ostaviti nepotpun fajl, koji će naredno učitavanje odbiti.

Funkcija ucitaj_red koristi fgets i uklanja preostali deo predugačkog reda, da naredni unos ne bi pročitao njegov nastavak. unesi_broj koristi strtol, proverava da je ceo unos broj i da pripada traženom opsegu. Time slova umesto broja ne izazivaju slučajan izbor opcije za izlaz. Pokazivači i povratne vrednosti poziva za alokaciju i otvaranje datoteke proveravaju se pre upotrebe.

2.4 Provera rada i ograničenja

Izvršeno je 13 funkcionalnih provera u odvojenim privremenim folderima. Testovi su poređivali stvarni izlaz i zapisanu datoteku sa očekivanim ishodom. Sve provere su prošle. Test oštećenih datoteka dodatno obuhvata sedam različitih neispravnih formata.

Provera	Ishod
FIFO upis, citanje, prikaz i praznjenje reda	Prošlo
Izmena postojeće datoteke bez zauzimanja novog bloka	Prošlo
Brisanje i odbijanje citanja nepostojeće datoteke	Prošlo
Pun disk, izmena na punom disku i ponovna upotreba bloka	Prošlo
Rezim samo citanje odbija upis i brisanje, dozvoljava citanje	Prošlo
Kvar zaustavlja zahtev bez izmene diska; oporavak omogućava upis	Prošlo
Cuvanje i učitavanje diska između dva pokretanja	Prošlo
Sedam oštećenih formata ne menjaju prethodno stanje	Prošlo
Učitavanje nepostojeće datoteke	Prošlo
Neispravan meni, vrsta, predugacak naziv i tekst	Prošlo
Maksimalna dozvoljena dužina naziva i sadržaja	Prošlo
Učitavanje diska je blokirano dok postoje zahtevi u redu	Prošlo
Kraj ulaza tokom unosa zahteva	Prošlo

Ograničenja modela su jedan proces, sekvencijalna obrada, pet blokova i jedna datoteka po bloku. Nema stvarnog upravljačkog programa, prekida, paralelnih niti, direktorijuma ni zaštite memorije. Funkcije slojeva dele podatke o modelu: drugi sloj čita nazive iz niza disk radi mapiranja, dok četvrti sloj izvršava promene zahtevane operacijom. Administrativne funkcije prikaza i učitavanja neposredno koriste model. Zato je ovo demonstracija logičkog toka i odgovornosti, a ne potpuno izolovana implementacija slojevitog jezgra.

Dalje proširenje moglo bi da izdvoji svaki sloj u poseban C modul, sakrije niz diska iza interfejsa i uvede više blokova po datoteci. Za sada se zadržava mali broj funkcija i opcija kako bi se svaka operacija mogla pratiti i objasniti na praktičnoj odbrani.

ZAKLJUČAK

Slojevita organizacija razdvaja odgovornosti i omogućava da se zahtev prati kroz više nivoa apstrakcije. Simulacija to prikazuje konkretnim promenama podataka: upis zauzima blok, čitanje vraća njegov sadržaj, a brisanje oslobađa prostor. Provera dozvole i simulirani kvar pokazuju da obrada može prestati u različitim slojevima bez izmene diska. Povezana lista, grananja, petlje i stvarno čuvanje slike diska povezuju teorijski model sa zahtevanim elementima C programiranja.

LITERATURA

- [1] R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau, Operating Systems Three Easy Pieces, Arpaci-Dusseau Books, 2023. <https://pages.cs.wisc.edu/~remzi/OSTEP/> (pristupljeno 22. 9. 2026).
- [2] Apple Developer, About Developing for Mac, Mac Technology Overview. https://developer.apple.com/library/archive/documentation/MacOSX/Conceptual/OSX_Technology_Overview/About/About.html (pristupljeno 22. 9. 2026).
- [3] Apple Developer, Kernel Architecture Overview, Kernel Programming Guide. <https://developer.apple.com/library/archive/documentation/Darwin/Conceptual/KernelProgramming/Architecture/Architecture.html> (pristupljeno 22. 9. 2026).
- [4] Microsoft Learn, Windows Kernel Mode HAL Library. <https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/windows-kernel-mode-hal-library> (pristupljeno 22. 9. 2026).
- [5] Microsoft Learn, User Mode and Kernel Mode. <https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/user-mode-and-kernel-mode> (pristupljeno 22. 9. 2026).
- [6] Microsoft Learn, End User I O Requests and File Objects. <https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/end-user-i-o-requests-and-file-objects> (pristupljeno 22. 9. 2026).
- [7] The Linux Kernel documentation, Overview of the Linux Virtual File System. <https://docs.kernel.org/filesystems/vfs.html> (pristupljeno 22. 9. 2026).
- [8] The Linux Kernel documentation, Concepts overview, Memory Management. <https://docs.kernel.org/6.2/admin-guide/mm/concepts.html> (pristupljeno 22. 9. 2026).
- [9] The Linux Kernel documentation, Driver Model. <https://docs.kernel.org/next/driver-api/driver-model/index.html> (pristupljeno 22. 9. 2026).
- [10] Microsoft Learn, Types of Windows Drivers. <https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/types-of-windows-drivers> (pristupljeno 22. 9. 2026).
- [11] E. W. Dijkstra, The Structure of the THE-Multiprogramming System, Communications of the ACM, 11(5), 341–346, 1968.
- [12] D. L. Parnas, On the Criteria To Be Used in Decomposing Systems into Modules, Communications of the ACM, 15(12), 1053–1058, 1972. DOI: 10.1145/361598.361623.